

# WEF1VO Web Fundamentals

## Vorlesungsunterlagen

Wolfgang Hochleitner  
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

## Vorlesung 10

# JavaScript DOM – Document Object Model

### 10.1 Allgemeines

Client-Side JavaScript ermöglicht es, aus statischen HTML-Webseiten dynamische und interaktive Dokumente zu kreieren. Dabei auf den Inhalt von Webseiten (also auf die einzelnen Elemente) zugreifen zu können, ist die wichtigste Eigenschaft in diesem Zusammenhang. Der Zugriff auf diese Inhalte erfolgt über das bereits in Vorlesung 9 vorgestellte `document`-Objekt und ist genau spezifiziert. Die Regeln und Methoden für den Zugriff sind im so genannten *Document Object Model*, kurz DOM, festgelegt.

Das *DOM* bezeichnet also eine Schnittstelle (engl. „API – application programming interface“) für den geregelten Zugriff auf hierarchische Markup-Dokumente (wie HTML oder XML). Die WHATWG (und zuvor auch das W3C) spezifiziert eine standardisierte Version des DOM, die in allen modernen Browsern gut unterstützt ist. Die Geschichte des DOM zeigt jedoch, dass dies nicht immer der Fall war.

#### 10.1.1 Versionen und Geschichte

Man spricht grob von vier unterschiedlichen DOM-Ausprägungen:

- Legacy DOM (DOM Level 0),
- Intermediate DOMs (IE4 DOM und Netscape 4 DOM),
- W3C DOM (DOM Level 1, 2, 3 & 4),
- WHATWG DOM Living Standard.

Netscape führte mit Version 2 ein DOM ein, das den Zugriff auf einige wenige Elemente auf der Seite ermöglichte. Darunter waren Links, Bilder und Formularelemente. Dieses DOM ist heute als *Legacy DOM* oder *DOM Level 0* bekannt, kommt allerdings in modernen Anwendungen nicht mehr zum Einsatz.

Mit Internet Explorer 4 brachte Microsoft eine eigene DOM-Variante heraus, die erstmals Zugriff auf alle Elemente der Seite ermöglichte. Sie wird heute in der Regel als IE4 DOM bezeichnet. Netscape – damals mit Microsoft mitten im Browserkrieg – entwickelte eine eigenständige, konkurrierende Variante zu der von Microsoft. Während das IE4 DOM zumindest nach wie vor der letzten Internet Explorer Versionen funktioniert,

war Netscapes Gegenstück eine Sackgasse und kam nur in Netscape 4 zum Einsatz. Da beide DOM-Varianten heutzutage aber de facto veraltet sind und nicht mehr zum Einsatz kommen, wird gesammelt von sogenannten *Intermediate DOMs* gesprochen. Auf sie wird auch nicht näher eingegangen.

Schließlich gibt es noch das bereits angesprochene, vom W3C standardisierte DOM, welches als *W3C DOM* bezeichnet wird. Da es in vier große (aufeinander aufbauende) Teile untergliedert ist, finden sich auch oft die Bezeichnungen *DOM Level 1*, *DOM Level 2*, *DOM Level 3* oder *DOM 4*. Level 1 enthält dabei die Kernaspekte des DOM-Scriptings (DOM Core), welche für diese Vorlesung relevant sind (siehe Abschnitt 10.2). Level 2, 3 und 4 erweitern das DOM um zusätzliche Funktionalitäten.

Seit 2019 wurde die kontinuierliche Weiterentwicklung des DOM – ähnlich wie bei HTML – an die WHATWG im Rahmen des *DOM Living Standards* übergeben.

## 10.2 W3C DOM und DOM Living Standard

Das *W3C DOM* ist eine offizielle, ursprünglich vom W3C verabschiedete Schnittstelle, die es Scriptsprachen erlaubt, dynamisch den Inhalt von Dokumenten zu verändern. Sie war in drei bestehende Ebenen gegliedert. Jede Ebene baute auf der vorhergehenden auf und erweitert den Funktionsumfang. Mittlerweile wird das DOM als WHATWG Living Standard (ähnlich wie HTML) kontinuierlich weiterentwickelt, die W3C Recommendations sind seit Ende 2020 als überholt (engl. „superseded“) eingestuft.

### 10.2.1 Ebenen des W3C DOM

*DOM Level 1* [7] war seit dem Jahr 1998 eine Recommendation und gliederte sich in zwei große Teile:

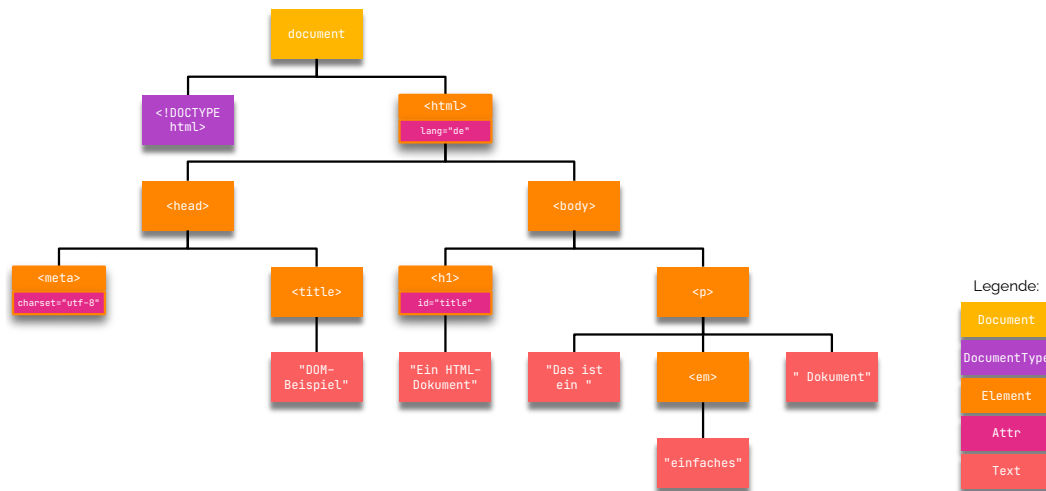
- **DOM Core Level 1:** Stellte die generellen Funktionalitäten zur Verfügung, die benötigt werden, um Elemente in der Dokumentenstruktur zu finden oder zu erzeugen. Diese sind sprachunabhängig, d.h. funktionieren sowohl für HTML als auch für alle XML-basierten Sprachen.
- **DOM HTML Level 1:** Stellte HTML-spezifische Funktionalitäten für den Umgang mit HTML-Elementen bereit.

*DOM Level 2* baute auf Level 1 auf und erweiterte sowohl die Core Funktionalitäten [8] als auch die HTML-Schnittstellen [10]. Zusätzlich wurde noch ein eigenes Event-System ergänzt [9], mit dem auf Aktionen im Browser reagiert werden kann, sowie ein eigenes Stylesheet Objektmodell [11], mit dem auf Stylesheets zugegriffen werden kann. Die meisten Spezifikationen waren seit 2000 eine Recommendation, die HTML-Erweiterung war es seit 2003.

*DOM Level 3* brachte weitere Ergänzungen für den Core [12], die im Jahr 2004 zur Recommendation erhoben wurden.

### 10.2.2 DOM und HTML Living Standards

2015 wurde vom W3C *DOM Level 4* mit weiteren Ergänzungen als Recommendation veröffentlicht. In den darauffolgenden Jahren wurde jedoch beschlossen, die kontinuierliche Arbeit am DOM der WHATWG zu übertragen, weshalb die letztgültige Variante des



**Abbildung 10.1:** Am Beginn der Baumstruktur steht das `document`-Objekt. Darunter entfaltet sich der Baum mit derselben hierarchischen Struktur, die durch den HTML-Code vorgegeben ist.

DOM nun im so genannten *DOM Living Standard* [4] abgebildet wird. Dies entspricht dem DOM Core und Event-Systemen aus den W3C Standards.

Die DOM HTML-Spezifika finden sich nun im *HTML Living Standard* [5] bei der Definition der jeweiligen Elemente und Attribute.

### 10.3 Repräsentation der Dokumentstruktur

HTML-Dokumente weisen eine hierarchische Struktur von ineinander verschachtelten Tags auf, wobei das `<html>`-Element immer das so genannte Wurzelement (auch Root-Element genannt) darstellt. Das Document Object Model repräsentiert diese Hierarchie in einer Baumstruktur. Die einzelnen Elemente in diesem Baum (HTML-Tags, Text und Attribute) werden *Knoten* genannt. Im DOM werden diese als Objekte vom Typ `Node` repräsentiert.

Dazu zunächst ein Beispiel. Ausgehend von folgendem HTML-Markup soll der entstandene DOM-Baum betrachtet werden.

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <meta charset="utf-8">
5     <title>DOM-Beispiel</title>
6   </head>
7   <body>
8     <h1 id="title">Ein HTML-Dokument</h1>
9     <p>Das ist ein <em>einfaches</em> Dokument</p>
10  </body>
11 </html>

```

Die entstandene Baumstruktur ist in Abbildung 10.1 ersichtlich.

Aus dem Beispiel wird klar, dass es nicht nur den generischen `Node`, sondern mehrere

Typen von Knoten gibt. Diese sind wiederum eigene Objekte, die aber von `Node` erben, d. h. jedes spezifischere Objekt ist auch immer ein `Node`.

- **Document**: Dieser Typ repräsentiert das gesamte Dokument und stellt die Wurzel des Baumes dar.
- **DocumentType**: Repräsentiert den `<!DOCTYPE>` eines Dokuments.
- **Element**: Element-Knoten repräsentieren in diesem Fall alle HTML-Elemente (`<h1>`, `<p>`, `<em>` etc.). In einem XML-Dokument werden XML-Elemente repräsentiert.
- **Attr**: Elemente von diesem Typ beinhalten Attribute, die – wie von HTML bekannt – einen Namen und einen Wert haben.
- **Text**: Umfasst Knoten, die jegliche Form von Text beinhalten. Eine Ausnahme bilden Kommentare. Diese werden durch den eigenen Knotentyp `Comment` repräsentiert.

Bei der Vorstellung der DOM-Gliederung in den Abschnitten 10.2.1 und 10.2.2 wurde in *DOM Core* und *DOM HTML* unterschieden. Die soeben vorgestellten Typen sind dem DOM Core, d. h. dem DOM Living Standard [4] zuzuordnen, da sie völlig allgemein bezeichnet sind. Elemente, Attribute, Text – all dies gibt es in jeder hierarchischen Markupsprache. Um nun aber diese Elemente aber spezifisch als HTML-Elemente zu klassifizieren, wird eine genauere Kategorisierung benötigt. Diese liefert das DOM HTML, also der HTML Living Standard [5]. In ihm sind Knotentypen speziell für HTML definiert bzw. allgemeine Typen erweitert. So gibt es:

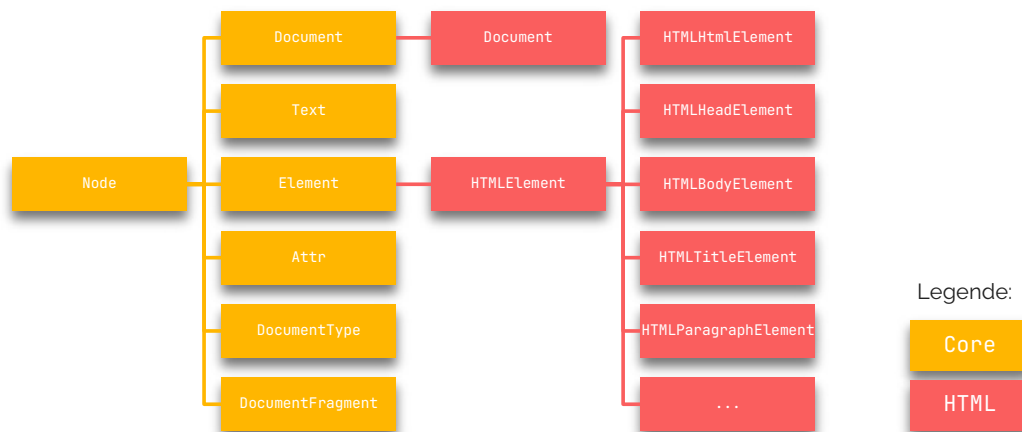
- **Document** (eine Erweiterung des **Document**-Typs um HTML-spezifische Inhalte).
- **HTMLElement** (die spezifischere Form von **Element**).

Für **Text** und **Attr** gibt es keine eigenen Knotentypen, da diese in HTML nicht anders funktionieren als in anderen Markupsprachen. Dafür hat aber der Knotentyp für **HTMLElement** eine Reihe noch spezifischerer Untertypen und zwar einen für jedes HTML-Element, das eigene Attribute besitzt (also mehr als nur die globalen Attribute):

- **HTMLHtmlElement** (repräsentiert `<html>`),
- **HTMLHeadElement** (repräsentiert `<head>`),
- **HTMLBodyElement** (repräsentiert `<body>`),
- **HTMLTitleElement** (repräsentiert `<title>`),
- **HTMLParagraphElement** (repräsentiert `<p>`),
- ...

Abbildung 10.2 stellt die Zusammenhänge der einzelnen Knotentypen im DOM grafisch dar.

Neben **Document**, das das gesamte Dokument repräsentiert, existiert auch noch **DocumentFragment**. Dabei handelt es sich um eine leichtgewichtige **Document**-Version, die keinen Elternknoten besitzt. Sie wird oft zur Performance-Optimierung verwendet, um Teilbäume im Speicher vorzubereiten, bevor sie in das sichtbare Dokument eingehängt werden. Mehr zum Erzeugen von Inhalten im nun folgenden Abschnitt 10.4.



**Abbildung 10.2:** Das Zusammenspiel zwischen Core- und HTML-DOM. Ein `<p>`-Element ist vom Typ `HTMLParagraphElement`, dieses Element ist wiederum eine spezifischere Version von `HTMLElement`, welches eine spezifischere Ausprägung von `Element` ist. Ganz am Anfang der Hierarchie steht `Node`. Jeder Knoten in einem DOM-Baum ist also prinzipiell vom Typ `Node`, wird aber dann schlussendlich von einem Subtypen genau spezifiziert.

## 10.4 Manipulation der Dokumentstruktur

Nachdem nun erörtert wurde, dass ein HTML-Dokument im DOM durch einen Baum, bestehend aus Knoten, repräsentiert wird, sollen nun die Möglichkeiten aufgezeigt werden, mit denen dieser Baum manipuliert werden kann. Manipulation bezeichnet das Hinzufügen, Verändern oder Löschen von Knoten, also das Verändern des HTML-Dokuments, das ja durch diese Knoten repräsentiert wird.

Die Funktionalität zur Manipulation der Struktur steckt in den Knoten selbst. Diese sind in JavaScript einfache Objekte, d. h. sie haben Eigenschaften (von denen Informationen über den Knoten abgefragt oder auch gesetzt werden können) und Methoden (mit denen grundlegende Änderungen wie z. B. das Erzeugen eines neuen Elements durchgeführt werden können). Allgemeine Eigenschaften und Methoden, die für alle Sprachen (HTML, XML etc.) gelten, finden sich bei Objekten im Core-Teil der Hierarchie (`Node`, `Document`, `Element`, `Attr`). Informationen, die nur für HTML relevant sind, können von den Objekten aus dem HTML-DOM (`HTMLElement`, `HTMLFormElement` etc.) abgefragt werden. Nachfolgend werden diese Eigenschaften und Methoden nach Objekten gruppiert vorgestellt.

### Node

Die Eigenschaften beim Typ `Node` liefern Informationen über den Knoten sowie den Kontext, in dem er sich befindet. Die dazugehörigen Methoden ermöglichen das Einfügen und Entfernen (aber nicht Erzeugen und Löschen) von neuen Knoten in den Knotenbaum. Es folgen die am häufigsten benötigten, MDN<sup>1</sup> bietet in seiner Auflistung noch

<sup>1</sup><https://developer.mozilla.org/en-US/docs/Web/API/Node>

einige weitere, weniger oft gebrauchte Eigenschaften und Methoden.

**Eigenschaften:** Folgende wichtige Eigenschaften sind über **Node** verfügbar. Zur Referenz sei folgendes HTML-Fragment gegeben:

```
1 <div><h1 id="title">Überschrift</h1><p>Hallo</p></div>
```

- **nodeName**: Name des Knotens bei **Element**- und **Attr**-Knoten. z. B. **div** oder **id**, **Text**-Knoten haben immer den Namen **#text**.
- **nodeValue**: Wert/Inhalt des Knotens bei **Text**- und **Attr**-Knoten. z. B. **Hallo** oder **title**. **null** für alle anderen Knoten.
- **textContent**: Der reine Textinhalt eines Knotens und dessen Kindknoten. z. B. **Hallo** für **<p>** oder **Überschrift Hallo** für **<div>**.
- **nodeType**: Der Typ des jeweiligen Knotens in Form einer Zahl: 1 für **Element**, 3 für **Text**, 8 für **Comment** oder 9 für **Document**.
- **firstChild**: Der erste Kindknoten. z. B. **<h1>** für das Element **<div>**.
- **lastChild**: Der letzte Kindknoten. z. B. **<p>** für das Element **<div>**.
- **nextSibling**: Das nächste Element auf der gleichen Ebene z. B. **<p>** für das Element **<h1>**.
- **previousSibling**: Das vorige Element auf der gleichen Ebene z. B. **<h1>** für das Element **<p>**.
- **parentNode**: Der Elternknoten. z. B. **<div>** für das Element **<h1>**.
- **parentElement**: Der Elternknoten, wenn dieser ein Element ist. **<div>** für **<h1>** und **<p>**.
- **childNodes**: Ein Array, das alle Kindknoten beinhaltet. z. B. **<h1>** und **<p>** für das Element **<div>**.

**Methoden:** Folgende Methoden sind **Node** zugeordnet:

- **insertBefore(newChild, refChild)**: Fügt einem Element den neuen Kindknoten **newChild** vor dem bestehenden Knoten **refChild** ein.
- **replaceChild(newChild, oldChild)**: Ersetzt aus Sicht eines Knotens den Kindknoten **oldChild** durch **newChild**.
- **appendChild(newChild)**: Hängt einem bestehenden Knoten den neuen Knoten **newChild** als letzten Kindknoten dazu.
- **removeChild(oldChild)**: Löscht einem Element seinen Kindknoten **oldChild**.
- **hasChildNodes()**: Gibt zurück, ob ein Element Kindknoten besitzt.
- **contains(node)**: Gibt zurück, ob **node** ein Unterknoten ist.
- **cloneNode(deep)**: Erstellt eine identische Kopie eines Knotens. Wird für **deep** der Wert **true** übergeben, werden auch alle Unterknoten mit kopiert, sonst nur der aktuelle.

Diese Eigenschaften und Methoden werden immer auf einem konkreten Knoten aufgerufen:

```
1 const theDiv = document.querySelector("div");
2
```

```

3 console.log("Name des Knotens:", theDiv.nodeName); // Ausgabe: "DIV"
4 console.log("Typ des Knotens:", theDiv.nodeType); // Ausgabe: 1
5
6 const h1 = theDiv.firstChild;
7 console.log("Erstes Kind (h1):", h1); // Gibt das <h1>-Element in der Konsole aus
8
9 const p = theDiv.lastChild;
10 console.log("Letztes Kind (p):", p); // Gibt das <p>-Element aus
11
12 // Ersetzen
13 theDiv.replaceChild(p, h1);
14 console.log("Nach replaceChild (h1 ist weg):", theDiv.innerHTML);
15
16 // Löschen
17 theDiv.removeChild(p);
18 console.log("Nach removeChild (leer):", theDiv.innerHTML);
19
20 console.log("Hat noch Kinder?", theDiv.hasChildNodes()); // Ausgabe: false

```

Damit `firstChild` hier auch tatsächlich das `<h1>`-Element zurückgibt, dürfen, wie im Snippet ersichtlich, keine Zeilenumbrüche im HTML-Markup vorkommen. Diese erzeugen ansonsten einen Textknoten in dem nur der Zeilenumbruch und allfällige Leerzeichen oder Tabulatoren für die Einrückung enthalten sind. Dieser Knoten wäre aber dann in diesem Fall erste Kindknoten des `<div>`-Elements.

## Document

Das Objekt `Document` erweitert `Node` und stellt bereits einige konkretere Methoden zur Verfügung. Während mittels `Node` nur Elemente in den DOM-Baum eingefügt oder gelöscht werden können, bietet `Document` nur die Möglichkeiten, neue Knoten überhaupt zu erzeugen und auch bestimmte Knoten zu finden. MDN<sup>2</sup> stellt auch hier eine vollständige Auflistung zur Verfügung.

**Eigenschaften:** `Document` bietet Zugriff auf einige allgemeine Eigenschaften:

- `URL` bzw. `documentURI`: Der aktuelle URL des Dokuments.
- `characterSet`: Die verwendete Zeichenkodierung, meist UTF-8.
- `contentType`: Der Content-Type der Datei, z. B. `text/html` für HTML-Dateien oder `application/xml` für XML-Dateien.
- `compatMode`: Der vom Browser verwendete Modus. Ist ein DOCTYPE vorhanden, wird `CSS1Compat` zurückgegeben, sonst `BackCompat`.

**Methoden:** Folgende wichtige Methoden stellt `Document` zur Verfügung:

- `getElementById(id)`: Wählt das Element aus, dessen ID-Attribut den Wert `id` besitzt.
- `getElementsByTagName(tagName)`: Gibt alle Elemente, mit dem übergebenen Tagnamen `tagName` zurück (z. B. alle `<p>`-Elemente auf der Seite). Diese Methode liefert immer eine Sammlung an Elementen (`HTMLCollection`), auch wenn es nur ein Element mit diesem Tagnamen gibt.

<sup>2</sup><https://developer.mozilla.org/en-US/docs/Web/API/Document>



- `getElementsByClassName(classNames)`: Gibt alle Elemente mit den im Argument `classNames` angegebenen Klassen zurück. Mehrere Klassen werden durch Leerzeichen getrennt spezifiziert. Gibt auch eine (`HTMLCollection`) zurück.
- `querySelector(selector)`: Gibt das erste Element retour, das dem übergebenen CSS-Selektor entspricht.
- `querySelectorAll(selector)`: Gibt eine Liste an Elementen (`NodeList`) retour, die einem CSS-Selektor entsprechen.
- `createElement(tagName)`: Erzeugt ein neues Element (einen Element-Knoten) mit dem Namen `tagName` (z. B. ein `<div>`-Element). Bevor es im Dokument erscheint, muss es mit den Methoden von `Node` (z. B. über `appendChild()`) noch in den DOM-Baum gehängt werden.
- `createTextNode(data)`: Erzeugt einen neuen Text-Knoten mit dem Inhalt `data`.
- `createAttribute(name)`: Erzeugt ein neues Attribut `name`, aber ohne Wert, das auch nicht an ein Element gehängt ist. Die Methoden dazu werden bei `Element` vorgestellt.

Diese Methoden werden immer mit dem `document`-Objekt aufgerufen. Als Beispiel sei wieder ein HTML-Fragment gegeben:

```
1 <div id="content">
2   <p class="red underline">Red and Underlined</p>
3 </div>
```

Auf diesem werden verschiedene Operationen ausgeführt:

```
1 const content = document.getElementById("content");
2 console.log("Gefundenes Element (ID):", content);
3
4 const bodyList = document.getElementsByTagName("body");
5 const body = bodyList[0];
6 console.log("Gefundenes Body-Element:", body);
7
8 const redUnder = document.getElementsByClassName("red underline");
9 console.log("Anzahl Elemente mit Klassen:", redUnder.length);
10
11 const pInDiv = document.querySelector("div p");
12 console.log("QuerySelector Ergebnis:", pInDiv);
13
14 // Erstellen
15 const newDiv = document.createElement("div");
16 console.log("Neu erstelltes DIV:", newDiv); // Zeigt <div></div> (noch leer)
17
18 const text = document.createTextNode("Hallo Welt!");
19 console.log("Text-Knoten:", text);
20
21 const idAttr = document.createAttribute("id");
22 console.log("Attribut-Knoten:", idAttr);
```

Ein wichtiger Hinweis zur Unterscheidung: Methoden wie `getElementsByTagName()` liefern eine sogenannte *live* `HTMLCollection`, die sich automatisch aktualisiert, sobald sich der DOM-Baum ändert (z. B. wenn neue Elemente hinzugefügt werden). Im Gegensatz dazu gibt `querySelectorAll()` eine *statische* `NodeList` zurück, die wie ein Schnappschuss den Zustand exakt zum Zeitpunkt der Abfrage behält und spätere Änderungen an der Struktur nicht widerspiegelt.

### Document (HTML-Spezifika)

Da die grundsätzlichen Methoden von `Document` sehr allgemein gehalten sind (also genauso für XML-Dokumente funktionieren), wird `Document` im Browser noch um solche erweitert, die spezifisch für HTML-Dokumente wichtig sind. Ebenso gibt es einige interessante Eigenschaften. Als Beispiel dient das folgende Fragment:

```
1 <form action="mailto:me@example.com">
2   <label for="wef">Web Fundamentals</label>
3   <input type="radio" name="favorite" value="WEF1" id="wef"><br>
4   <label for="mco">Modern Code</label>
5   <input type="radio" name="favorite" value="MCO1" id="mco"><br>
6   <label for="gec">Game Engine Concepts</label>
7   <input type="radio" name="favorite" value="GEC1" id="gec"><br>
8   <button type="submit">Senden</button>
9 </form>
```

**Eigenschaften:** Die hier zur Verfügung gestellten Eigenschaften wurden quasi bereits in Vorlesung 9 rund um das `document`-Objekt vorgestellt. Neben dem direkten Zugriff auf Struktur-Elemente (`head`, `body`) gibt es auch Zugriff auf Listen von spezifischen HTML-Elementen sowie Browser-Informationen.

- **title:** Der Titel des aktuellen Dokuments, wie er im `<title>`-Element spezifiziert wurde.
- **head:** Das `<head>`-Element der Seite.
- **body:** Das `<body>`-Element der Seite.
- **forms:** Eine automatisch aktualisierte Liste (`HTMLCollection`) aller `<form>`-Elemente im Dokument.
- **images:** Eine Liste aller `<img>`-Elemente im Dokument.
- **cookie:** Ermöglicht den Zugriff auf die Cookies, die zu diesem Dokument gehören.
- **location:** Ein Objekt, das Informationen zur aktuellen URL enthält und auch zum Navigieren verwendet werden kann.

**Methoden:** Bei den Methoden ist vor allem die Selektion nach dem `name`-Attribut wichtig, welches häufig in Formularen vorkommt. Die Methoden zum direkten Schreiben in den Dokumentenstrom gelten heute als veraltet („deprecated“) und sollten in modernen Anwendungen nicht mehr verwendet werden.

- **getElementByName(name):** Wählt die Element mit den entsprechenden `name`-Attributen aus. Liefert immer ein Array.
- **open(), write(), close():** Diese Methodengruppe erlaubt es, einen Dokumentenstrom zu öffnen, Text direkt hineinzuschreiben und ihn wieder zu schließen. *Achtung:* Da diese Methoden das Dokument blockieren oder komplett überschreiben können, werden sie heute fast ausschließlich zur Fehlersuche oder in sehr spezifischen Altlasten verwendet. Die Manipulation des DOM erfolgt stattdessen über das Erzeugen und Einhängen von Knoten.

Auch diese Eigenschaften und Methoden werden mit dem `document`-Objekt aufgerufen:

```
1 console.log("Seitentitel:", document.title);
2 console.log("Body-Referenz:", document.body);
3
4 const favorites = document.getElementsByName("favorite");
5 console.log("Gefundene Radio-Buttons:", favorites.length);
6
7 // HTML-Spezifische Collections (automatisch indiziert)
8 console.log("Anzahl Formulare:", document.forms.length);
9 console.log("Anzahl Bilder:", document.images.length);
10
11 // Zugriff auf das erste Formular direkt über den Index
12 console.log("Erstes Formular:", document.forms[0]);
13
14 // Cookies lesen und schreiben
15 // (Funktioniert meist nur, wenn die Seite über einen Server geladen wird)
16 document.cookie = "lecture=WEF1; SameSite=Lax";
17 console.log("Alle Cookies:", document.cookie);
18
19 // URL-Informationen
20 console.log("Aktuelle URL:", document.location.href);
21
22 // Schreiben ins Dokument (veraltet, nicht mehr verwenden, daher auskommentiert)
23 // document.open();
24 // document.write("<p>Hallo Welt!</p>");
25 // document.close();
```

## Element

**Element** erweitert wie **Document** das Objekt **Node** und führt zusätzliche Eigenschaften und Methoden im Umgang mit Elementen ein. MDN<sup>3</sup> enthält wiederum eine vollständige Auflistung aller Eigenschaften und Methoden.

**Eigenschaften:** Diese Eigenschaften sind bei jedem **Element**-Knoten abfragbar:

- **tagName:** Der Name des Elements (z. B. **p**).
- **className:** Der Wert des **class**-Attributs eines Elements.
- **classList:** Gibt eine **DOMTokenList** zurück, die eine komfortable Schnittstelle zur Manipulation der CSS-Klassen bietet. Im Gegensatz zur manuellen Bearbeitung des Strings in **className** bringt sie Methoden wie **add()**, **remove()**, **toggle()** (Umschalten) und **contains()** (Prüfen) mit.
- **id:** Der Wert des **id**-Attributs
- **innerHTML:** Liefert den Inhalt eines HTML-Elements, inklusive aller Nachfahr-Elemente. Bei **textContent** (Eigenschaft von **Node**) werden die Elemente entfernt und nur der Text zurückgeliefert.
- **firstElementChild:** Liefert den ersten Kindknoten, der ein Element ist.
- **lastElementChild:** Liefert den letzten Kindknoten, der ein Element ist.
- **nextElementSibling:** Selektiert den nächsten Geschwisterknoten, der ein Element ist.
- **previousElementSibling:** Selektiert das vorangehende Geschwisterelement.

---

<sup>3</sup><https://developer.mozilla.org/en-US/docs/Web/API/Element>

Methoden: Die Methoden, die `Element` zur Verfügung stellt, dienen meist zum Umgang mit Attributen, jedoch können auch Unterelemente damit gesucht werden.

- `getAttribute(name)`: Gibt den Wert des Attributs `name` zurück.
- `setAttribute(name, value)`: Setzt (erzeugt) ein Attribut mit dem Namen `name` und Wert `value`.
- `removeAttribute(name)`: Löscht das Attribut `name`.
- `hasAttribute(name)`: Überprüft, ob ein Element das Attribut `name` besitzt.
- `setAttributeNode(name)`: Hängt an ein Element einen Attributknoten `name`, der zuvor z. B. mit `createAttribute(name)` erzeugt wurde.
- `getAttributeNode(name)`: Fragt den Attributknoten `name` vom Element ab.
- `remove()`: Entfernt das Element aus dem Baum, in dem es hängt.

Darüber hinaus sind hier nochmals alle Selektionsmethoden aus der `Document`-Schnittstelle, also etwa `querySelector()` oder `getElementsByClassName()` vorhanden. Sie funktionieren gleich, allerdings selektieren Sie nur Elemente, die Kinder desjenigen sind, auf dem die Methode ausgeführt wird. Sie suchen also nicht im gesamten Dokument.

Die Eigenschaften und Methoden werden direkt auf Elemente (Knoten vom Typ `Element`) angewandt, dieses wird hier zunächst erzeugt:

```
1 const heading = document.createElement("h1");
2 heading.textContent = "Überschrift";
3
4 heading.setAttribute("id", "important");
5 console.log("ID gesetzt:", heading.id); // Ausgabe: "important"
6
7 heading.id = "normal";
8 console.log("ID geändert:", heading.getAttribute("id")); // Ausgabe: "normal"
9
10 heading.className = "blue";
11 console.log("Klasse gesetzt:", heading.className);
12
13 heading.classList.add("underline");
14 console.log("Klassenliste:", heading.classList); // DOMTokenList ["blue", "underline"]
15
16 document.body.appendChild(heading);
17 console.log("Element in den Body eingehängt:", heading);
```

## HTMLElement

`HTMLElement` stellt spezifische Eigenschaften vor, die nur für HTML gelten. Sie decken sich mit den Universalattributen, die es bei jedem HTML-Element gibt. Dieser Zugriff auf `Attribute` ist in der Regel einfacher als mit `getAttribute()` und `setAttribute()` und wird daher bevorzugt.

Eigenschaften: Die folgenden Universalattribute sind vorhanden:

- `title`: Das `title`-Attribut.
- `lang`: Das `lang`-Attribut.
- `dir`: Das `dir`-Attribut (Textleserichtung).

- **style**: Das **style**-Attribut.
- **accessKey**: Der zugeordnete Access-Key.
- **tabIndex**: Der zugeordnete Tab-Index.

Diese Eigenschaften werden immer auf Elemente angewandt, wiederum wird zu-nächst eines erzeugt.

```
1 const divElem = document.createElement("div");
2 divElem.textContent = "Hallo Welt!"; // Textinhalt setzen
3 divElem.lang = "de"; // Sprache setzen
4 divElem.style.marginTop = "2em"; // Stil-Eigenschaft setzen
5
6 console.log("Konfiguriertes Element:", divElem);
7 // In der Konsole kann man das Element aufklappen und die Styles prüfen
8
9 document.body.appendChild(divElem);
```

### Subtypen von HTMLElement

Die restlichen HTML-Attribute sind – je nachdem welchem Element sie angehören – bei den jeweiligen Objekten angeführt. Einige Beispiele:

- **HTMLLinkElement**: Eigenschaften: **href**, **target**, **rel** etc.
- **HTMLFormElement**: Eigenschaften: **action**, **method**, **name** etc.
- **HTMLInputElement**: Eigenschaften: **name**, **value**, **size** etc. Methoden: **blur()**, **focus()** etc.
- **HTMLImageElement**: Eigenschaften: **src**, **alt**, **width**, **height** etc.

Diese Eigenschaften werden immer auf Elemente angewandt. Hier wird ein **<input>**-Element erstellt und einige Eigenschaften verändert:

```
1 // 1. Container-Formular erstellen
2 const form = document.createElement("form");
3
4 // 2. Das Input-Element erstellen
5 const input = document.createElement("input");
6
7 // Typ ändern
8 input.type = "password";
9
10 // Autocomplete setzen (z.B. für Passwort-Manager)
11 input.autocomplete = "new-password";
12
13 input.placeholder = "Geheimcode eingeben";
14 input.value = "123456";
15
16 // Deaktivieren
17 input.disabled = true;
18
19 console.log("Aktueller Wert:", input.value);
20 console.log("Ist deaktiviert:", input.disabled);
21
22 // 3. Zusammenbauen
23 form.appendChild(input);
24 document.body.appendChild(form);
```

## Attr

Die Attributknoten vom Typ `Attr` sind sehr einfach aufgebaut und enthalten lediglich zwei Eigenschaften, die ein Attribut beschreiben:

Eigenschaften: Ein Attribut kann zwei Eigenschaften haben:

- **name**: Der Name des Attributs.
- **value**: Der Wert des Attributs.

Diese Eigenschaften werden immer auf Attributknoten angewandt:

```
1 const divElem = document.createElement("div");
2 divElem.textContent = "An ordinary div";
3
4 const titleAttr = document.createAttribute("title");
5 titleAttr.value = "Some Container";
6 console.log("Attribut-Wert:", titleAttr.value);
7
8 divElem.setAttributeNode(titleAttr);
9 console.log("HTML mit Attribut:", divElem.outerHTML);
10 // Ausgabe z.B.: <div title="Some Container">An ordinary div</div>
11
12 document.body.appendChild(divElem);
```

Die explizite Erstellung von `Attr`-Knoten über `createAttribute()` und das Einhängen via `setAttributeNode()` verdeutlicht zwar die interne Knoten-Struktur des DOM, ist im Alltag jedoch unnötig umständlich. In der modernen Webentwicklung wird fast ausschließlich die wesentlich einfachere Methode `setAttribute("name", "value")` oder der direkte Zugriff über Eigenschaften (z. B. `element.id = "..."`) verwendet.

## 10.5 Events

Bisher wurde JavaScript-Code in den Beispielen vorwiegend linear betrachtet: Der Browser liest das Dokument, stößt auf ein `<script>`-Element und führt den Inhalt sofort aus. Dies geschieht synchron während des Ladevorgangs (Parsing).

Interaktive Anwendungen funktionieren jedoch anders. Aktionen sollen oft nicht sofort beim Laden, sondern erst zu einem späteren, unbestimmten Zeitpunkt stattfinden – etwa wenn Benutzer\*innen einen Button anklicken, eine Eingabe tätigen oder die Fenstergröße ändern. Zudem erfordert die DOM-Manipulation zwingend, dass die entsprechenden Elemente bereits vollständig geladen und im Baum verfügbar sind, bevor das Skript darauf zugreift.

Für Entwickler\*innen ist es daher essentiell zu wissen, *wann* solche Situationen eintreten. Zu diesem Zweck generiert der Browser immer dann, wenn eine relevante Aktion stattfindet, sogenannte *Events* (deutsch: „Ereignisse“). Dies ermöglicht es, den Code von der linearen Ausführung zu entkoppeln und gezielt auf Interaktionen zu reagieren.

### 10.5.1 Arten von Events

Im Browser werden die unterschiedlichsten Arten von Ereignissen generiert. Viele davon entstehen aufgrund von Eingaben der Benutzer\*innen (z. B. Mausklicks oder Tastatureingaben), andere Events werden implizit ausgelöst, etwa wenn die Seite vollständig

geladen ist oder ein Formular abgeschickt wurde. Ähnlich wie beim DOM gibt es auch hier verschiedene Schnittstellen, die oberste und generellste ist dabei **Event** aus dem DOM Living Standard [4]. Je nach Typ gibt es dann spezifischere Subtypen, die etwa aus der W3C UI Events Spezifikation [14] stammen. Um die Maus als Eingabegerät zu generalisieren, wurden Pointer Events [13] geschaffen, die neben der Maus auch Stifte und Touch repräsentieren und daher zu bevorzugen sind. Wenn ein Event verarbeitet wird (siehe dazu Abschnitt 10.5.2) steht das Event-Objekt selbst zur Verfügung. Es enthält die genauen Informationen über das jeweilige Ereignis (z. B. wo genau geklickt wurde) und hat einen spezifischen Typ – hat also je nach Ereignisart andere Eigenschaften und Methoden.

Tabelle 10.1 gibt eine Übersicht über die in der DOM-Spezifikation definierten Events und ihre Typen. Es sei angemerkt, dass viele andere Spezifikationen, die im Umfeld von HTML und JavaScript angesiedelt sind, weitere Events definieren. Auf diese wird hier jedoch nicht eingegangen.

### 10.5.2 Verarbeiten von Events

Die generierten Ereignisse alleine geben jedoch nur Aufschluss darüber, *dass* etwas Bestimmtes aufgetreten ist. Ohne einen Mechanismus, darauf zu reagieren, ist diese Information aber nur bedingt nützlich. JavaScript stellt jedoch Mechanismen zur Verfügung, um genau dies zu tun. Der generelle Ablauf, auf einen Event zu reagieren sieht dabei wie folgt aus:

1. **Element auswählen, bei dem das Ereignis stattfindet:** Im HTML oder mittels einer DOM-Methode wird das Element gesucht bzw. ausgewählt, auf dem eine Benutzer\*inneninteraktion stattfindet bzw. wo vom Browser implizit Ereignisse generiert werden, z. B. ein Button, der geklickt werden soll.
2. **Event spezifizieren, der abgefangen werden soll:** Unter Angabe eines der Events, die in Tabelle 10.1 aufgelistet wurden, wird angegeben, worauf reagiert werden soll, z. B. auf einen Mausklick, der auf den zuvor spezifizierten Button ausgeführt wird.
3. **Verarbeitung in einer Funktion angeben:** Bei der Verarbeitung eines Ereignisses wird immer eine Funktion aufgerufen, in dieser wird nun spezifiziert, was genau passieren soll, z. B. eine Alert-Box mit einer Nachricht ausgeben, wenn auf den Button mit der Maus geklickt wurde.

Diese drei Schritte sind immer dieselben, wenn auf Events reagiert werden soll. Es gibt jedoch unterschiedliche Ansätze, wie Events abgefangen werden können:

- als HTML-Event-Handler,
- als DOM-Event-Handler, oder
- als DOM-Event-Listener.

#### Verarbeitung mittels HTML-Event-Handler

HTML-Event-Handler sind nichts anderes als HTML-Attribute, den den Namen des Events mit vorangestelltem `on` tragen. Für einen `click`-Event heißt das Attribut also `onclick`, für den `blur`-Event `onblur` usw.

**Tabelle 10.1:** Im DOM spezifizierte Events und ihre Typen.

| Event                   | Beschreibung                                                                                       | Objektyp             |
|-------------------------|----------------------------------------------------------------------------------------------------|----------------------|
| <b>abort</b>            | Das Laden einer Ressource (z. B. Bild) wurde abgebrochen.                                          | <b>UIEvent</b>       |
| <b>blur</b>             | Ein Element hat den Fokus verloren.                                                                | <b>FocusEvent</b>    |
| <b>change</b>           | Ein Formular-Element hat einen geänderten Wert erhalten.                                           | <b>Event</b>         |
| <b>click</b>            | Maus- bzw. Pointerклик.                                                                            | <b>PointerEvent</b>  |
| <b>dblclick</b>         | Maus-Doppelклик.                                                                                   | <b>MouseEvent</b>    |
| <b>DOMContentLoaded</b> | Das aktuelle Dokument (d. h. sein DOM-Baum) wurde fertig geladen, jedoch nicht externe Ressourcen. | <b>Event</b>         |
| <b>error</b>            | Fehler beim Laden einer Ressource.                                                                 | <b>UIEvent</b>       |
| <b>focus</b>            | Ein Element hat den Fokus erhalten.                                                                | <b>FocusEvent</b>    |
| <b>focusin</b>          | Ein Element hat den Fokus erhalten.                                                                | <b>FocusEvent</b>    |
| <b>focusout</b>         | Ein Element hat den Fokus verloren.                                                                | <b>FocusEvent</b>    |
| <b>keydown</b>          | Taste wird gedrückt gehalten.                                                                      | <b>KeyboardEvent</b> |
| <b>keyup</b>            | Taste wurde losgelassen.                                                                           | <b>KeyboardEvent</b> |
| <b>load</b>             | Dokument ist fertig geladen.                                                                       | <b>UIEvent</b>       |
| <b>pointerdown</b>      | Ein Pointer (Maus, Pen, Touch etc.) wurde gedrückt.                                                | <b>PointerEvent</b>  |
| <b>pointerenter</b>     | Der Pointer wird über ein Element bewegt.                                                          | <b>PointerEvent</b>  |
| <b>pointerleave</b>     | Der Pointer wird von einem Element wegbewegt.                                                      | <b>PointerEvent</b>  |
| <b>pointermove</b>      | Der Pointer wurde bewegt.                                                                          | <b>PointerEvent</b>  |
| <b>pointerout</b>       | Der Pointer wird von einem Element oder einem seiner Kinder wegbewegt.                             | <b>PointerEvent</b>  |
| <b>pointerover</b>      | Der Pointer wird über ein Element oder eines seiner Kinder bewegt.                                 | <b>PointerEvent</b>  |
| <b>pointerup</b>        | Der Pointer wurde losgelassen.                                                                     | <b>PointerEvent</b>  |
| <b>reset</b>            | Ein Formular wurde zurückgesetzt.                                                                  | <b>Event</b>         |
| <b>resize</b>           | Die Browserfenstergröße wurde geändert.                                                            | <b>UIEvent</b>       |
| <b>scroll</b>           | Es wurde nach unten oder oben gescrollt.                                                           | <b>UIEvent</b>       |
| <b>select</b>           | Text wurde selektiert.                                                                             | <b>UIEvent</b>       |
| <b>submit</b>           | Ein Formularfeld wurde abgeschickt.                                                                | <b>Event</b>         |
| <b>unload</b>           | Das aktuelle Dokument wurde weggeladen, d. h. eine neue Seite aufgerufen.                          | <b>Event</b>         |

Als Wert des Attributs folgt der JavaScript-Code, der aufgerufen werden soll, in Form einer Funktion. Angenommen, im Code ist eine Funktion `alertMe()` definiert, so kann diese bei einem Klick auf einen Button folgendermaßen aufgerufen werden:

```
1 <button id="button1" type="button" onclick="alertMe()">Button 1</button>
```

Diese Art und Weise, Event-Handler zuzuweisen, erscheint zwar effizient, vermischt aber Struktur und Logik einer Website, d. h. HTML und JavaScript. Daher gilt sie mittlerweile als veraltet und sollte nicht mehr angewandt werden.



### Verarbeitung mittels DOM-Event-Handler

Um diese beiden Dinge sauber zu trennen, kann der Event-Handler auch rein im JavaScript-Code zugewiesen werden. Dazu wird zunächst das HTML-Element mit der entsprechenden DOM-Methode gesucht und diesem dann der Event-Handler als Eigenschaft zugewiesen. Auch hier lauten die Eigenschaften wie der Event-Name mit vorangestelltem `on`, also `onclick`, um den `click`-Event abzufangen.

```
1 <button id="button2" type="button">Button 2</button>
```

Die Zuweisung über JavaScript geschieht wie folgt:

```
1 const btn2 = document.getElementById("button2");  
2 btn2.onclick = alertMe;
```

Zunächst wird der Button mit der Methode `getElementById()` gesucht und in einer Variable gespeichert. Dann kann mit der einfachen Punkt-Syntax eine neue Eigenschaft, in diesem Fall `onclick` gesetzt werden. Der Wert ist die Funktion, die aufgerufen werden soll.

Vorsicht: Die Zuweisung muss *ohne* abschließende Klammern erfolgen. Dies garantiert, dass die gesamte Funktion dem `onclick`-Handler zugewiesen und diese erst beim eigentlichen Klick aufgerufen wird. Würde stattdessen `btn2.onclick = alertMe();` geschrieben, so würde `alertMe()` sofort ausgeführt werden und der Rückgabewert der Funktion (in diesem Fall nur `undefined`) würde der `onclick`-Eigenschaft zugewiesen.

Wichtig ist ebenfalls, dass zum Zeitpunkt der Zuweisung das HTML-Element bereits existieren muss. Dies bedeutet, die Zuweisung muss in einem `<script>`-Block ganz am Ende des `<body>`-Elements oder im `<head>` mit dem `defer`-Attribut geschehen.

Diese Art der Event-Handler-Zuweisung hat jedoch einen Nachteil: Es lässt sich nur eine Funktion pro Event zuweisen. Um diese Einschränkung zu umgehen, kann eine *anonyme Funktion* verwendet werden. Eine solche Funktion hat keinen Namen und existiert nur zum Zeitpunkt ihres Aufrufs. Mit ihr können aber mehrere weitere Funktionsaufrufe gekapselt werden, sodass etwa `alertMe()` zwei Mal hintereinander aufgerufen werden könnte:

```
1 btn2.onclick = function () {  
2   alertMe();  
3   alertMe();  
4 };
```

### Verarbeitung mittels DOM-Event-Listener

Die flexibelste Art, Events zu verarbeiten sind die im DOM Living Standard [4] definierten Methoden zum Hinzufügen und Entfernen von Event-Listnern:

- `addEventListener(event, function)`,
- `removeEventListener(event, function)`.

Um damit beim Klick auf einen Button die Funktion `alertMe()` aufzurufen, werden zunächst wieder ein Button spezifiziert

```
1 <button id="button3" type="button">Button 3</button>
```

und dann in der Methode der Eventname (ohne „on“) und der Name der aufzurufenden Funktion (wieder ohne Klammern) angegeben:

```
1 const btn3 = document.getElementById("button3");
2 btn3.addEventListener("click", alertMe);
```

Event-Listener haben den Vorteil, dass beliebig viele verarbeitete Funktionen für einen Event zugewiesen werden können, dazu wird `addEventListener()` einfach mehrmals mit demselben Event aufgerufen. Auch das Entfernen von Event-Listnern ist ein großer Vorteil. In diesem Skriptum kommen daher nur Event-Listener zum Einsatz.

### 10.5.3 Standardverhalten von Events unterbinden

Bei vielen Interaktionen im Browser werden automatisch Events ausgelöst. Wird etwa ein Link angeklickt, erfolgt ein `click`-Event, der den Browser dazu veranlasst, dem im `href`-Attribut spezifizierten URL zu folgen. Wird nun der Link jedoch verwendet, um mittels JavaScript selbst etwas auszuführen, so ist oft dieses Standardverhalten unerwünscht und muss unterdrückt werden.

Folgendes Beispiel illustriert diese Situation, es sei diese Funktion definiert:

```
1 function showURL() {
2   const link = document.querySelector("a");
3   window.alert(link.href);
4 }
```

Sie nimmt das erste `<a>`-Element der Seite und gibt den Wert dessen `href`-Attributs in einem Dialogfenster aus. Der Inhalt des `<body>`-Elements sieht wie folgt aus:

```
1 <a href="https://www.fh-ooe.at/">Ein Link</a>
2 <script>
3   const link = document.querySelector("a");
4   link.addEventListener("click", showURL);
5 </script>
```

Zunächst wird ein Link erstellt, der auf eine Seite verweist, danach wird ein Event-Listener auf diesen Link zugewiesen, der die Funktion `showURL()` beim Klick auf den Link aufruft.

Wird nun diese HTML-Datei im Browser geöffnet und auf den Link geklickt, so erscheint zunächst eine Dialogbox, die den Inhalt des `href`-Attributs (also den URL) ausgibt. Wird diese Box geschlossen, wird die Standardaktion des Browsers ausgeführt und zum angegebenen URL gewechselt. Da es nur um die Ausgabe des Attributwerts geht, ist diese nicht erwünscht, es muss also die Standardaktion, d. h. das Folgen des Links, unterbunden werden.

Um dies zu erreichen, wird der Funktion das Event-Objekt (das von Browser standardmäßig erzeugt wird) als Parameter eingetragen. Dieses Objekt besitzt die Methode `preventDefault()`, die innerhalb der Funktion am Event-Objekt aufgerufen werden muss und das Standardverhalten unterbindet. `showURL()` muss also wie folgt aussehen:

```
1 function showURL(event) {
2   const link = document.querySelector("a");
3   window.alert(link.href);
4   event.preventDefault();
5 }
```

Nun zeigt der Browser das Dialogfenster an, wechselt aber danach nicht auf den URL sondern bleibt auf der aktuellen Seite.

### 10.5.4 Zugriff auf das Listener-Element

Bei genauerer Betrachtung des letzten Beispiels fällt auf, dass das gewünschte Element (der Link) eigentlich zweimal gesucht wird: einmal beim Zuweisen des Event-Listeners und ein zweites Mal innerhalb der Funktion `showURL()`. Auch wenn dies per se nicht falsch ist, so ist es nicht sonderlich elegant und performant (zwei Abfragen im DOM-Baum) und macht die Funktion `showURL()` auch nicht wiederverwendbar. Sie ist fest an einen spezifischen Selektor gebunden. Werden weitere Links eingefügt, müsste die Funktion für jeden Link neu angepasst oder kopiert werden.

Um die Funktion universell zu gestalten, wird die Möglichkeit benötigt zu sagen „Gib den Inhalt des `href`-Attributs von *demjenigen* Element aus, dessen Event-Listener gerade die Funktion aufgerufen hat“. Die Lösung hierfür ist der Zugriff über das `event`-Objekt, genauer gesagt über die Eigenschaft `event.currentTarget`. Diese Eigenschaft verweist immer zuverlässig auf das Element, an dem der Event-Listener registriert wurde – unabhängig davon, ob klassische Funktionen oder Arrow-Functions verwendet werden.

Zunächst wird die Funktion entsprechend geändert. Sie selektiert nun kein Element mehr selbst, sondern greift direkt auf das auslösende Element zu:

```
1 function showURL(event) {  
2   // Zugriff auf das Element, das den Listener ausgelöst hat  
3   window.alert(event.currentTarget.href);  
4   event.preventDefault();  
5 }
```

Im HTML-Markup werden nun zwei Hyperlinks platziert. Um den Event-Listener an alle Links zu binden, wird eine `for...of`-Schleife verwendet. Damit können jederzeit neue Links hinzugefügt werden und von der Funktion berücksichtigt werden.

```
1 <a href="https://www.fh-ooe.at/">Ein Link</a>  
2 <a href="https://orf.at/">Noch ein Link</a>  
3 <script>  
4   const links = document.querySelectorAll("a");  
5  
6   // Jeden Link durchlaufen und Listener anhängen  
7   for (const link of links) {  
8     link.addEventListener("click", showURL);  
9   }  
10 </script>
```

Die Funktion `showURL()` ist nun universell einsetzbar: sie zeigt automatisch den URL jenes Links an, der gerade angeklickt wurde, ohne dass die Funktion selbst wissen muss, um welchen Link es sich handelt.

### 10.5.5 Event-Listener Beispiele

Zur Demonstration von einigen Event-Listnern folgen nun drei Beispiele, von denen vor allem das erste eine wichtige Funktionalität zeigt.

Auf vollständiges Laden der Seite warten

Ein häufiges Problem bei der DOM-Manipulation ist das Timing. Da der Browser HTML-Dateien linear von oben nach unten abarbeitet, existieren Elemente im `<body>`

noch nicht, wenn ein `<script>`-Block im `<head>` bereits ausgeführt wird. Ein Zugriff via `document.querySelector()` würde ins Leere laufen.

Früher (und in vielen älteren Tutorials) wurde dieses Problem gelöst, indem entweder das `<script>` ganz ans Ende des `<body>` platziert wurde oder in JavaScript explizit auf Events gewartet wurden. Heute steht, wie in Vorlesung 9 gezeigt, mit dem `defer`-Attribut eine elegantere Lösung zur Verfügung, die bevorzugt werden sollte. Dennoch sollen die Events zum Laden der Seite hier nicht unerwähnt bleiben.

**DOMContentLoaded:** Technisch gesehen wartet das `defer`-Attribut auf den Zeitpunkt, an dem der Browser das Event `DOMContentLoaded` feuern würde. Dieses Event markiert den Moment, in dem das reine HTML vollständig verarbeitet und in den DOM-Baum umgewandelt wurde.

Möchte man JavaScript-Code direkt innerhalb der HTML-Datei, also im `<script>`-Element, ohne externe Datei im `<head>` schreiben, funktioniert `defer` nicht. In diesem Fall muss manuell auf das Event `DOMContentLoaded` gewartet werden, um zu wissen, wann der DOM-Baum erstellt wurde:

```
1 document.addEventListener("DOMContentLoaded", function () {  
2   // Dieser Code läuft erst, wenn der DOM bereit ist  
3   console.log("HTML ist fertig geparsed!");  
4 });
```

In der modernen Praxis wird jedoch fast immer mit externen Dateien und `defer` gearbeitet, weshalb der explizite Einsatz von `DOMContentLoaded` selten geworden ist.

**load:** Wichtig ist die Abgrenzung zum `load`-Event beim `window`-Objekt. Während `defer` und `DOMContentLoaded` feuern, sobald die *Struktur* (HTML) steht, wartet `load`, bis wirklich alles geladen ist – also auch große Bilder, Stylesheets und externe Skripte. Dies wird nur benötigt, wenn das Skript von visuellen Eigenschaften abhängt, die erst nach dem Laden von Assets bekannt sind (z. B. „Wie hoch ist dieses Bild in Pixeln wirklich?“).

**Beispiel:** Das folgende Beispiel demonstriert zeigt Events. Die JavaScript-Datei wird im `<head>` der HTML-Datei eingebunden. Auf das Attribut `defer` wird bewusst verzichtet. In der Datei befindet sich ein `<div>`-Element, in das im Anschluss Status-Meldungen geschrieben werden sowie ein Bild und ein Button.

```
1 <!DOCTYPE html>  
2 <html lang="de">  
3   <head>  
4     <meta charset="utf-8">  
5     <title>Unterschied: DOMContentLoaded vs. load</title>  
6     <script src="domcontentloaded-load.js"></script>  
7   </head>  
8   <body>  
9     <div id="status">Warte auf Events...</div>  
10      
11    <button id="actionButton">Klick mich</button>  
12  </body>  
13 </html>
```

In der JavaScript-Datei werden zwei Funktionen definiert. `initInteractions()`, die Event-Listener hinzufügt, um beim Button und beim Bild auf Klicks zu reagieren. Ebenso wird die Meldung in der Statusbox aktualisiert:

```
1 function initInteractions() {
2   const statusBox = document.querySelector("#status");
3   const btn = document.querySelector("#actionButton");
4   const img = document.querySelector("#demoImage");
5
6   statusBox.textContent = "DOMContentLoaded gefeuert: Struktur ist bereit.";
7   statusBox.style.backgroundColor = "#e0f7fa"; // Hellblau
8
9   // Listener anhängen möglich, da das HTML-Element existiert
10  btn.addEventListener("click", function () {
11    window.alert("Button funktioniert, obwohl das Bild evtl. noch lädt!");
12  });
13
14  img.addEventListener("click", event => {
15    console.log("Bild wurde angeklickt:", event.currentTarget.src);
16  });
17 }
```

Die zweite Funktion heißt `finalizePage()` und zeigt in der Statusbox die Größe des Bildes an. Dies funktioniert erst, wenn das Bild auch tatsächlich geladen ist:

```
1 function finalizePage() {
2   const statusBox = document.querySelector("#status");
3   const img = document.querySelector("#demoImage");
4
5   // Erst jetzt kennen wir die echte Größe des Bildes
6   const width = img.naturalWidth;
7   const height = img.naturalHeight;
8
9   statusBox.textContent += ` | Load gefeuert: Bild geladen (${width}x${height}px).`;
10  statusBox.style.backgroundColor = "#c8e6c9"; // Hellgrün
11  statusBox.style.border = "2px solid green";
12 }
```

Damit nun beide Funktionen zur richtigen Zeit ausgeführt werden, werden die entsprechenden Event-Listener registriert:

- `DOMContentLoaded` beim `document`-Objekt und
- `load` beim `window`-Objekt.

```
1 document.addEventListener("DOMContentLoaded", initInteractions);
2 window.addEventListener("load", finalizePage);
```

**Anonyme Funktionen:** Oft soll bei einem Event-Listener (insbesondere beim Warten auf das Laden der Seite) mehr als nur eine Funktion angegeben werden, die aufgerufen werden soll, wenn der Event stattfindet. Dafür könnte eine eigene Funktion erstellt werden, in der wiederum mehrere andere Funktionen aufgerufen werden. Alternativ kann aber auch eine anonyme Funktion erstellt und dem Listener zugewiesen werden. Anonyme Funktionen existieren nur an Ort und Stelle ihrer Definition und haben auch keinen Namen (daher anonym). Sie sind einfach wie folgt aufgebaut:

```

1 function () {
2   // Inhalt der anonymen Funktion
3 }

```

Soll nun vor der `initInteractions()`- und der `finalizePage()`-Funktion jeweils noch eine Dialogbox geöffnet werden (um z. B. zu erfahren, dass der jeweilige Event vom Browser ausgelöst wurde), so kann dies in einer anonymen Funktion (klassisch oder als Arrow-Funktion) verpackt werden:

```

1 document.addEventListener("DOMContentLoaded", function () {
2   window.alert("DOMContentLoaded-Event ausgelöst!");
3   initInteractions();
4 });
5
6 window.addEventListener("load", () => {
7   window.alert("load-Event ausgelöst!");
8   finalizePage();
9 });

```

### Absenden eines Formulars überprüfen

Ein Event-Listener für das `submit`-Event kann beim `<form>`-Element verwendet werden. Er wird beim Absenden des Formulars aufgerufen. `preventDefault()` ist auch hier interessant, denn es verhindert das Absenden des Formulars. Somit eignet sich der `submit`-Event bestens für eine Formularfeldüberprüfung mit JavaScript. Es ruft eine Funktion auf, welche die Eingaben aller Formularfelder überprüft. Sind alle Kriterien erfüllt (z. B. alle Pflichtfelder ausgefüllt und E-Mail-Adressen korrekt eingegeben), so wird das Formular wie erwartet verschickt. Sind noch Fehler in den Eingaben vorhanden, wird mittels `preventDefault()` das Absenden verhindert und es können etwa neben die jeweiligen Formularfelder Hinweise platziert werden, wo sich Fehler befinden.

Das Formularfeld kann wie folgt aussehen:

```

1 <form action="mailto:john.doe@johndoe.at" method="post"
2   enctype="text/plain" id="dataform">
3   <label for="name">Name:</label>
4   <input type="text" id="name">
5   <button type="submit">Absenden</button>
6 </form>

```

In JavaScript wird dann dem Formular der Listener zugewiesen:

```

1 // Merkt sich, ob Fehler aufgetreten sind
2 let hasErrors = false;
3
4 function isValid(event) {
5   // Validierung der Formularfelder und setzen von hasErrors auf true, wenn ein Fehler
6     passiert ist
7   document.querySelector("input").value === "" ? hasErrors = true : hasErrors =
8     false;
9
10  if (hasErrors) {
11    window.alert("Absenden verhindert");
12    event.preventDefault();
13  }
14 }

```

```
13
14 // Zuweisen der Funktion auf submit-Listener des Formulars
15 document.getElementById("dataform").addEventListener("submit", isValid);
```

### Auf Änderungen überprüfen

Während `click` für Buttons ideal ist, benötigen Formularelemente wie Dropdown-Menüs (`<select>`), Checkboxes oder Textfelder einen anderen Auslöser. Hier ist das `change`-Event das Mittel der Wahl. Es wird ausgelöst, sobald der Wert eines Elements geändert und die Änderung „abgeschlossen“ wurde (z. B. durch Drücken von Enter oder Verlassen des Feldes).

Besonders bei Dropdowns bietet das DOM einen sehr komfortablen Zugriff, denn es existiert eine Eigenschaft `value` beim `<select>`-Element, die immer den Wert der ausgewählten Option enthält. Es muss also nicht „händisch“ abgefragt werden, welches `<option>`-Element gewählt wurde.

Folgendes Formular mit einem Dropdown sei gegeben:

```
1 <form action="" method="post">
2   <label for="choice">Auswahl:</label>
3   <select id="choice">
4     <option value="A">Wert A</option>
5     <option value="B">Wert B</option>
6     <option value="C">Wert C</option>
7   </select>
8 </form>
```

Im `<head>` der HTML-Datei ist wieder eine JavaScript-Datei mit `defer` eingebunden. Deren Inhalt ist wie folgt:

```
1 function valChange(event) {
2   window.alert(`${event.currentTarget.value} wurde ausgewählt`);
3 }
4
5 document.getElementById("choice").addEventListener("change", valChange);
```

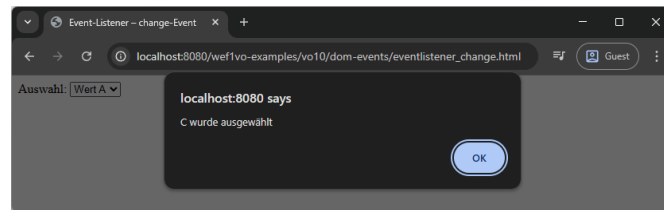
Nachdem `defer` sichergestellt hat, dass das HTML vollständig geladen ist, wird das `<select>`-Element über seine `id` gesucht. Seinem `change`-Listener wird die Funktion `valChange()` zugewiesen. Wann immer in der Selectbox etwas neues ausgewählt wird, wird diese Funktion aufgerufen.

`valChange()` gibt in einem Dialogfenster aus, welcher Wert ausgewählt wurde. Da der Event-Listener am `<select>`-Element angebracht ist, kann – wie in Abschnitt 10.5.4 gezeigt – über `event.currentTarget` dieses abgefragt werden. In der Eigenschaft `value` steht jener Wert aus der Option, die gewählt wurde. Hier ist dies entweder „A“, „B“ oder „C“.

Abbildung 10.3 zeigt das Ergebnis, nachdem Wert C ausgewählt wurde.

## 10.6 Weiterführende Literatur

Zur Arbeit mit dem DOM sind die deutschen JavaScript-Fachbücher [1, 3, 6] zu empfehlen, die dank ihrer kontinuierlichen Neuauflagen auch moderne DOM-Neuerungen



**Abbildung 10.3:** Sobald eine Option ausgewählt wird, kommt ein Dialogfenster und gibt dessen Wert aus. Das Fenster erscheint dabei noch bevor der Wert in der Selectbox umgestellt wird.

miteinbeziehen. Alle DOM-Spezialwerke sind mittlerweile zu alt, um empfohlen werden zu können.

Als Online-Referenz sei an dieser Stelle auch nochmals [2] erwähnt, welches zum Nachschlagen diverser JavaScript-Objekte hilfreich sein kann.



# Quellenverzeichnis

- [1] Philip Ackermann. *JavaScript. Das umfassende Handbuch*. 4. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. Nov. 2025 (siehe S. 10-22).
- [2] Mozilla Developer Network and individual contributors. *Mozilla Developer Network*. Jan. 2026. URL: <https://developer.mozilla.org/en-US/> (besucht am 19.01.2026) (siehe S. 10-23).
- [3] Thomas Theis. *Einstieg in JavaScript*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Juli 2024 (siehe S. 10-22).
- [4] Web Hypertext Application Technology Working Group. *DOM*. Living Standard. 15. Jan. 2026. URL: <https://dom.spec.whatwg.org/> (siehe S. 10-3, 10-4, 10-14, 10-16).
- [5] Web Hypertext Application Technology Working Group. *HTML*. Living Standard. 15. Jan. 2026. URL: <https://html.spec.whatwg.org/multipage/> (siehe S. 10-3, 10-4).
- [6] Jürgen Wolf. *HTML und CSS. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Aug. 2023 (siehe S. 10-22).
- [7] World Wide Web Consortium. *Document Object Model (DOM) Level 1 Specification Version 1.0*. W3C Recommendation. 1. Okt. 1998. URL: <https://www.w3.org/TR/REC-DOM-Level-1/> (siehe S. 10-2).
- [8] World Wide Web Consortium. *Document Object Model (DOM) Level 2 Core Specification Version 1.0*. W3C Recommendation. Superseded on 3 November 2020. 13. Nov. 2000. URL: <https://www.w3.org/TR/DOM-Level-2-Core/> (siehe S. 10-2).
- [9] World Wide Web Consortium. *Document Object Model (DOM) Level 2 Events Specification Version 1.0*. W3C Recommendation. Superseded on 3 November 2020. 13. Nov. 2000. URL: <https://www.w3.org/TR/DOM-Level-2-Events/> (siehe S. 10-2).
- [10] World Wide Web Consortium. *Document Object Model (DOM) Level 2 HTML Specification Version 1.0*. W3C Recommendation. Superseded on 3 November 2020. 9. Jan. 2003. URL: <https://www.w3.org/TR/DOM-Level-2-HTML/> (siehe S. 10-2).
- [11] World Wide Web Consortium. *Document Object Model (DOM) Level 2 Style Specification Version 1.0*. W3C Recommendation. Superseded on 3 November 2020. 13. Nov. 2000. URL: <https://www.w3.org/TR/DOM-Level-2-Style/> (siehe S. 10-2).

- [12] World Wide Web Consortium. *Document Object Model (DOM) Level 3 Core Specification Version 1.0*. W3C Recommendation. 7. Apr. 2004. URL: <https://www.w3.org/TR/DOM-Level-3-Core/> (siehe S. 10-2).
- [13] World Wide Web Consortium. *Pointer Events Level 2*. W3C Recommendation. 4. Apr. 2019. URL: <https://www.w3.org/TR/pointerevents2/> (siehe S. 10-14).
- [14] World Wide Web Consortium. *UI Events*. W3C Working Draft. 7. Sep. 2024. URL: <https://www.w3.org/TR/uevents/> (siehe S. 10-14).